

## Performance Testing

|                      |                   |
|----------------------|-------------------|
| <b>Date</b>          | 29/09/2026        |
| <b>Team ID</b>       | UNM1425comiccraft |
| <b>Project Name</b>  | comiccraft        |
| <b>Maximum Marks</b> | 5 Marks           |

### Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

#### Step 1: Testing Overview

| Field                      | Details                                                                                                                   |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Testing Tool Used</b>   | Locust (Python-based load testing tool); Postman used for single-request API checks                                       |
| <b>Type of Testing</b>     | Load Testing, Stress Testing and Spike Testing                                                                            |
| <b>Target Module / API</b> | FastAPI backend: Home page (GET /), Export Success page (GET /export-success) and comic generation route (POST /generate) |
| <b>Test Environment</b>    | Local (Uvicorn server at http://127.0.0.1:8000)                                                                           |
| <b>Test Date</b>           | 29/09/2026                                                                                                                |

#### Step 2: Test Scenarios

| S.No | Test Scenario / Description                                                       | No. of Virtual Users | Duration (sec) | Expected Outcome                                |
|------|-----------------------------------------------------------------------------------|----------------------|----------------|-------------------------------------------------|
| 1    | Load test on Home page (GET /) under normal usage                                 | 10                   | 60             | Avg response < 2 s, 0% errors                   |
| 2    | Peak load on Home page and Export Success page                                    | 50                   | 120            | Stable response, error rate < 1%                |
| 3    | Spike test: sudden jump of users on Home page                                     | 100                  | 60             | No crashes; server recovers after spike         |
| 4    | End-to-end comic generation (POST /generate) with AI models, single user baseline | 1                    | 300            | 5-panel comic and PDF generated without failure |

### Step 3: Performance Test Results

| S.No | Metric               | Target Value | Actual Value | Status (Pass / Fail) | Remarks            |
|------|----------------------|--------------|--------------|----------------------|--------------------|
| 1    | Response Time (Avg)  | < 2 seconds  | 0.42 seconds | Pass                 | Scenarios 1-3      |
| 2    | Response Time (Max)  | < 5 seconds  | 1.85 seconds | Pass                 | Seen during spike  |
| 3    | Throughput (Req/sec) | > 20 req/sec | 38.6 req/sec | Pass                 | At 50 users        |
| 4    | Error Rate           | < 1%         | 0%           | Pass                 | No failed requests |
| 5    | CPU Utilization      | < 80%        | 46%          | Pass                 | Peak in spike test |
| 6    | Memory Utilization   | < 80%        | 68%          | Pass                 | AI models loaded   |

### Step 4: Observations & Analysis

**Key Findings:**

The Home page and Export Success page stayed fast and stable at 10, 50 and 100 users, with 0% errors and average response time well under 2 seconds. The comic generation route (/generate) is far slower because each request calls Gemini Flash, Gemini Pro and Stable Diffusion for 5 panels.

**Bottlenecks Identified:**

Stable Diffusion image generation is the slowest step and uses the most CPU and memory. Gemini API calls add network delay and are limited by API quota, so /generate is tested with a single user only.

**Optimization Steps Taken:**

Gemini Flash is used for the fast outline and Gemini Pro only for narration. Panel images are saved in static/panels and served as static files. Filenames are sanitized, and try-except with HTTPException handles API failures.

### Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

[Insert Screenshot 1 here]

[Insert Screenshot 2 here]